LAB 02

Dibyasundar Das

3 August 2026

LAB 02

Design a Utility-Based Vacuum Cleaner Agent (m × n Grid) with Performance Metrics for fully observable world.

- Design a Vacuum Cleaner World of size **m × n** using a 2D matrix representation of cells.
- Place dirt randomly in the cells (each cell is either `0` = Clean or `1` = Dirty).
- Place the agent randomly in one of the cells.
- Unlike the partially observable version, this is a **fully observable environment** — the agent has **complete and accurate knowledge of the entire grid's true state at all times**, including cells it has not yet physically visited. There is no separate "belief"/"knowledge" grid and no `?` (unknown) state — the agent can directly see the dirt status of every cell from the start.
- The agent can move **Up, Down, Left, or Right**, restricted by grid boundaries.
- If the current cell is dirty, the agent must **vacuum** it before moving.
- This must be a **Utility-Based Agent** — instead of moving randomly or reactively, the agent evaluates the **utility of each possible action** using its complete knowledge of the grid and chooses the action that **maximizes utility** at every step.
- Because the environment is fully observable, the utility function can be computed **exactly** (no probability/belief estimation is required) — the agent always knows precisely where every dirty cell is and can compute true distances, true costs, and true expected rewards.
- The simulation should continue until every cell in the grid is actually clean, or a maximum number of iterations is reached.

Utility Function Design

- Since the environment is fully observable, the agent knows the exact location of every dirty cell at all times. This means utility can be computed exactly — no probability or belief estimation needed — using true shortest-path distances (BFS) over the known grid.
- Formula
  - For a candidate action a (either "vacuum" or a move to neighbor cell pos'):

```
U(a) =      VacuumReward(a)
          − DistanceToNearestDirty(a)
```

```
            - MovementCost(a)
            + DirtDensityNearby(a)
```

Where:

- `VacuumReward(a)` = fixed large reward (e.g., 10) if a is "vacuum" and current cell is dirty; 0 otherwise.
- `DistanceToNearestDirty(a)` = BFS shortest-path distance from the resulting position to the nearest dirty cell (0 if no dirt remains).
- `MovementCost(a)` = 1 if a is a move, 0 if a is vacuum (moving costs more than acting on current cell).
- `DirtDensityNearby(a)` = number of dirty cells within a small radius of the resulting position (rewards moving toward dirt clusters, not just the single nearest one).

Notation Explained

Symbol	Meaning
`D`	Dirty cell (visible from the start, fully observable)
`C`	Clean cell
`V`	Agent just vacuumed this cell
`A()`	Agent present, no action taken this step
`A(U)` / `A(D)` / `A(L)` / `A(R)`	Agent present, moved Up/Down/Left/Right

Suggested Functions

1. `create_environment(rows, cols)` – creates the 2D matrix representing the world's true state (fully visible to the agent).
2. `place_dirt(env)` – randomly places dirt in the grid.
3. `place_agent(rows, cols)` – randomly places the agent at a starting `(row, col)`.
4. `is_dirty(env, pos)` – checks the dirt status of a given cell (directly readable, since the world is fully observable).
5. `vacuum(env, pos)` – cleans the current cell.
6. `get_possible_moves(pos, rows, cols)` – returns valid moves from the current position.
7. `find_nearest_dirty_cell(env, pos, rows, cols)` – uses BFS/shortest-path search over the fully known grid to locate the nearest dirty cell and the distance to it.
8. `compute_utility(action, pos, env, rows, cols)` – computes the utility score of a candidate action (vacuum or move) using true distances and rewards.
9. `choose_best_action(pos, env, rows, cols)` – evaluates utility of vacuuming vs. each possible move and returns the action with maximum utility.

10. `all_clean(env)` – checks whether all cells in the true environment are clean (valid termination check here, since the environment is fully observable).
11. `compute_theoretical_minimum(env, start_pos, rows, cols)` – computes (or estimates) the minimum number of steps needed to clean all dirty cells, for the optimality ratio metric.
12. `update_metrics(metrics, action_taken, ...)` – updates the running performance-metric counters after each step.
13. `display_environment(env, agent_pos, action, step, rows, cols)` – prints the full grid state and the utility-based decision made at each step.
14. `display_performance_summary(metrics)` – prints the final performance report after the simulation ends.
15. `run_vacuum_world(rows, cols)` – main driver function integrating full-state utility computation, decision-making, metric tracking, and termination.

Design a Utility-Based Vacuum Cleaner Agent (m × n Grid) with Performance Metrics for Partially observable world.

- Design a Vacuum Cleaner World of size $m \times n$ using a 2D matrix representation of cells.
- Place dirt randomly in the cells (each cell is either `0` = Clean or `1` = Dirty) — this represents the **true, hidden state** of the environment.
- Place the agent randomly in one of the cells.
- The agent maintains its **own knowledge** of the world, separate from the true hidden environment. Initially, the agent knows nothing about any cell (including its own starting cell) until it **perceives** it.
- The agent can move **Up, Down, Left, or Right**, restricted by grid boundaries.
- If the current cell is dirty, the agent must **vacuum** it before moving.
- **Unlike a simple reflex agent**, this agent must be a **Utility-Based Agent** — instead of choosing moves randomly or by fixed rule, it must evaluate the **expected utility of each possible action** and choose the action that **maximizes utility** at every step.
- The agent will never know if a cell is clean or dirty unless it has physically visited (perceived) that cell.
- The simulation should continue until the agent's own knowledge confirms all cells are clean, or a maximum number of iterations is reached.